

Remedi Android Application Report

CSE461 Mobile Computing Project

Application Type: Medication Reminder and Health Management App

Technology Stack: Kotlin, XML, MVVM, Room Database, Navigation Component

Platform: Native Android Application

Prepared by:

Omar El Nabarawy 22-101123

Rokaya Radwan 22-101171

Jana Sherif 22-101234

Semester: Spring 2026

Contents

1	Problem Statement	3
2	System Overview	4
3	Team Members and Contributions	4
4	MVVM Architecture	5
4.1	Architecture Diagram	5
4.2	Layer Responsibilities	6
5	Room Database Schema	7
5.1	Main Entities	7
5.2	Database Relationship Diagram	7
5.3	User-Specific Medication Storage	8
6	Fragment Map and Navigation Flow	8
6.1	Fragment Map	8
6.2	Fragment Responsibilities	9
7	Implementation Details	10
7.1	Authentication Flow	10
7.2	Medication Flow	10
7.3	Reminder Flow	10
7.4	Bottom Navigation	10
8	Performance Findings	10
8.1	Database Performance	10
8.2	RecyclerView Performance	11
8.3	Configuration Changes	11
8.4	Memory and Lifecycle	11
9	Code Quality and Android Best Practices	12
9.1	Kotlin and XML	12
9.2	ViewBinding	12
9.3	MVVM Separation	12
9.4	Room and Coroutines	12
9.5	Navigation Component	12
9.6	SharedPreferences for Session Data	12
10	Challenges Encountered	13
10.1	Room Integration	13
10.2	User-Specific Data	13
10.3	Foreign Key Safety	13

10.4 Navigation Bar Design	13
10.5 Theme Compatibility	13
10.6 Database Versioning	13
11 Conclusion	14
12 GitHub REPO URL	14

1. Problem Statement

Many people, especially patients with chronic illnesses, elderly users, and users who take multiple medications daily, struggle to manage their medication schedules correctly. Missing a dose, taking a dose at the wrong time, or forgetting refill information can negatively affect health outcomes. In many cases, users rely on memory, handwritten notes, or separate alarm applications, which are not designed specifically for medication management.

The **Remedi** application addresses this problem by providing a mobile solution for managing medications, reminders, and user health-related settings. The app allows users to register an account, log in, add their medications, view medication lists, create reminders, and access profile settings. All important data is stored locally using Room Database, which allows the application to work without depending on an online server.

The target users of Remedi include individuals who take daily medication, patients managing multiple prescriptions, caregivers who help others manage treatment schedules, and users who want a simple health reminder system. A mobile application is appropriate for this problem because users usually carry their phones throughout the day, making reminders and medication tracking more accessible and practical.

The main goals of the application are:

- Help users store and manage personal medication information.
- Allow each registered user to access only their own medications.
- Provide a structured reminder schedule for medication intake.
- Maintain local data persistence using Room Database.
- Follow the MVVM architecture for clean separation of concerns.
- Provide a clear user interface using XML layouts and Android Jetpack components.

2. System Overview

Remedi is a native Android application built using Kotlin. The application follows the MVVM architectural pattern and uses XML for designing the user interface. Room Database is used for local data persistence, while the Android Navigation Component manages transitions between screens.

The application contains authentication screens, medication management screens, reminder scheduling screens, a dashboard, and a profile/settings screen. After registering, the user is redirected to the login screen. When the user logs in successfully, the app stores the logged-in user's session data using SharedPreferences. This session contains the user's ID, full name, and email. The logged-in user ID is then used to filter medication records so that each user only sees their own medications.

The main implemented features include:

- User registration and login using Room.
- Dashboard with personalized greeting.
- Medication creation and display.
- User-specific medication filtering through `userId`.
- Reminder creation linked to existing medications.
- Fixed bottom navigation bar for main screens.
- Profile/settings screen.
- Local data persistence through Room.

3. Team Members and Contributions

Team Member	Contribution
Jana Sherif	<i>UI/UX layouts, Room database, Dao and repository</i>
Omar El Nabarawy	<i>Project lead, UI/UX layouts, Room Database and ViewModel layer</i>
Rokaya Ramy	<i>UI/UX layouts, Room database, Dao and repository</i>

Table 1: Team members and their contributions

4. MVVM Architecture

The project follows the MVVM pattern, which separates the application into View, View-Model, Repository, DAO, and Database layers. This separation improves maintainability, testability, and code organization.

4.1. Architecture Diagram

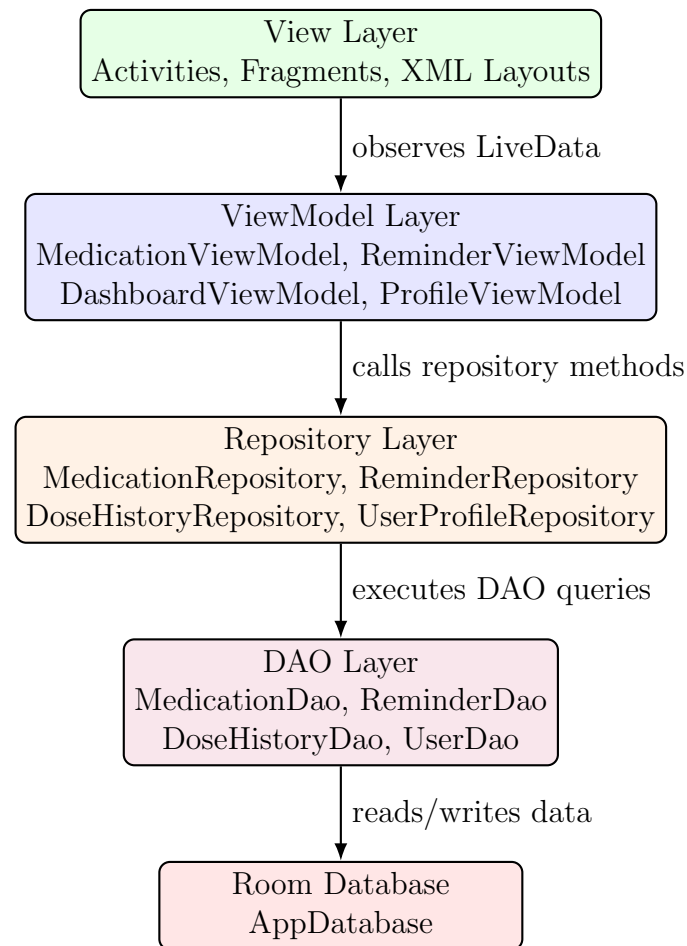


Figure 1: Remedi MVVM Architecture

4.2. Layer Responsibilities

View Layer: This layer includes fragments such as `LoginFragment`, `RegisterFragment`, `DashboardFragment`, `MedicationListFragment`, `TodayMedicationsFragment`, `AddEditMedicationFragment`, `ReminderScheduleFragment`, and `ProfileSettingsFragment`. The View layer displays data, receives user input, and observes `LiveData` from the View-Model. It does not directly contain database query logic.

ViewModel Layer: ViewModels hold UI-related data and survive configuration changes such as screen rotation. For example, `MedicationViewModel` exposes the user's medications as `LiveData` and provides functions for adding, updating, and deleting medication records. ViewModels call repositories instead of directly accessing DAOs.

Repository Layer: Repositories act as a clean API between ViewModels and DAOs. For example, `MedicationRepository` receives a `userId` and exposes only medications belonging to that user. This keeps database access organized and avoids placing SQL-related logic inside fragments.

DAO Layer: DAO interfaces contain SQL queries and database operations. Examples include inserting medications, retrieving medications for a specific user, counting medications, and reading reminders.

Room Database Layer: The `AppDatabase` class defines all Room entities and provides access to DAO interfaces. It represents the local SQLite database used by the app.

5. Room Database Schema

Room Database is used to persist user data locally. The schema includes users, medications, reminders, and user profile information.

5.1. Main Entities

Entity	Purpose	Important Fields
UserEntity	Stores registered users.	id, fullName, email, password
MedicationEntity	Stores medication records for each user.	id, userId, name, dosage, time, instruction, refillsLeft, color, createdAt
ReminderEntity	Stores reminder records linked to medications.	id, medicationId, medicineName, doseAmount, time, repeatType, isEnabled, status
DoseHistoryEntity	Stores medication dose history.	id, medicationId, medicineName, doseDetails, dateTime, status
UserProfileEntity	Stores user profile settings and preferences.	fullName, email, emergencyContact, notificationsEnabled, darkModeEnabled

5.2. Database Relationship Diagram

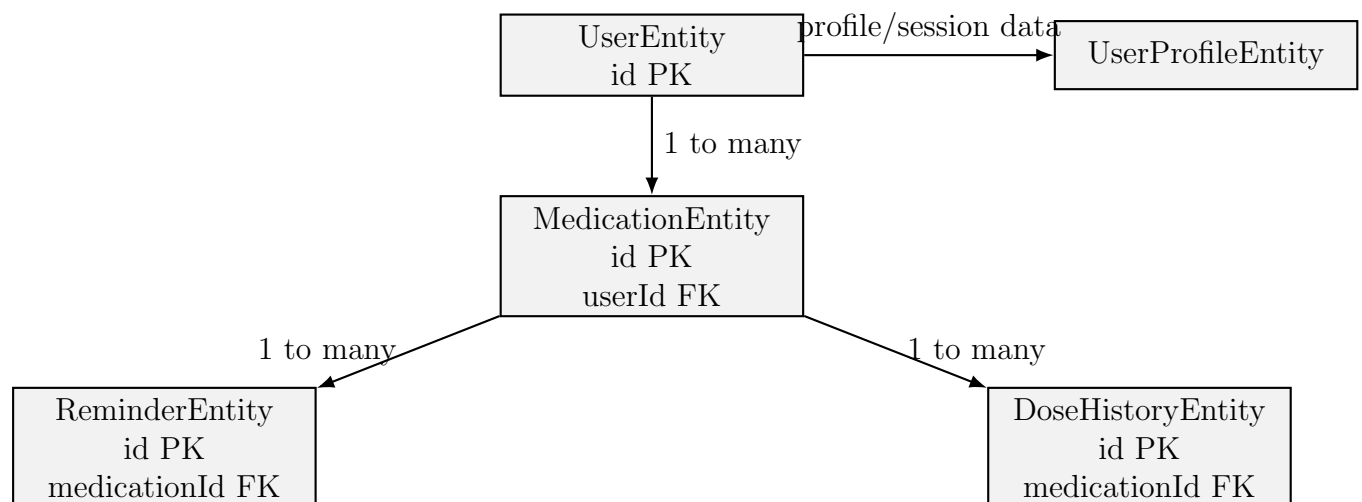


Figure 2: Room Database Relationships

5.3. User-Specific Medication Storage

A major improvement in the database design is the addition of `userId` to `MedicationEntity`. This means each medication belongs to a specific registered user. When a user logs in, the app stores the logged-in user's ID in `SharedPreferences`. Medication queries then use this ID to load only medications belonging to the current user.

Instead of loading all medications with:

```
SELECT * FROM medications
```

the app loads user-specific medications using:

```
SELECT * FROM medications WHERE userId = :userId ORDER BY id DESC
```

This prevents one user's medications from appearing in another user's account.

6. Fragment Map and Navigation Flow

The application uses the Android Navigation Component to manage movement between fragments. Navigation is defined in `nav_graph.xml`. The app starts at the welcome screen, then allows users to log in or register.

6.1. Fragment Map

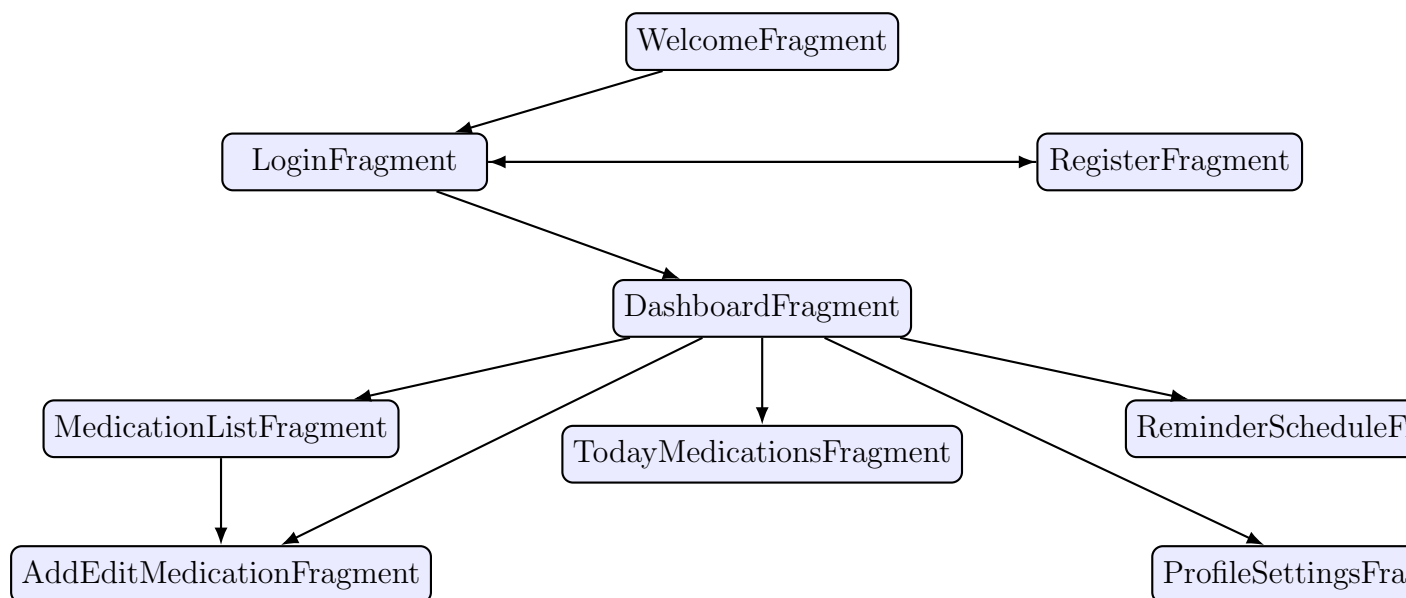


Figure 3: Fragment Navigation Map

6.2. Fragment Responsibilities

Fragment	Responsibility
WelcomeFragment	First screen shown to the user. Provides entry to login.
LoginFragment	Validates user credentials using Room and stores logged-in user session.
RegisterFragment	Saves a new user to Room, then returns the user to the login screen.
DashboardFragment	Shows personalized greeting, daily content, and today's medication section.
MedicationListFragment	Shows all medications belonging to the logged-in user.
TodayMedicationsFragment	Shows the current user's medications scheduled for today or currently available daily medications.
AddEditMedicationFragment	Allows the user to add medication data and saves it with the current user's ID.
ReminderScheduleFragment	Allows users to create reminders by selecting an existing medication.
ProfileSettingsFragment	Shows profile information and user settings.

7. Implementation Details

7.1. Authentication Flow

The application uses a local Room-based authentication system. When a user registers, a new row is inserted into the `users` table. After successful registration, the user is sent back to the login screen instead of being logged in automatically. This allows the login process to validate the saved credentials.

During login, the app queries the database for a user with the entered email and password. If a matching record exists, the user's information is saved in `SharedPreferences`. The saved session is then used throughout the app to identify the current user.

7.2. Medication Flow

When the user adds a medication, the app reads the current user's ID from `SharedPreferences`. The medication is then saved into Room with that `userId`. This ensures that each medication belongs to a specific user. Medication list screens use the same `userId` to retrieve only the current user's records.

7.3. Reminder Flow

Reminders are linked to medications using the `medicationId` foreign key. Instead of allowing users to type medication names freely, the reminder screen loads the user's existing medications into a spinner. The user selects a medication, and the reminder is saved using the selected medication's real ID. This prevents invalid reminder records and protects the relationship between medications and reminders.

7.4. Bottom Navigation

The bottom navigation bar is implemented in `MainActivity`. This is better than adding it separately to every fragment because it keeps the navigation fixed and avoids repeated XML code. The navigation bar includes Home, Medications, a central Add Medication button, Reminders, and Profile. It is hidden on screens where it would be inappropriate, such as login, register, welcome, and add/edit medication screens.

8. Performance Findings

The app was designed with performance and lifecycle awareness in mind. Room operations are executed using Kotlin coroutines and background dispatchers, preventing database operations from blocking the main UI thread.

8.1. Database Performance

Room provides efficient local data access because it is built on SQLite. Queries such as loading medications for a specific user are simple and indexed using `userId`. This improves filtering performance as the number of users and medications increases.

8.2. RecyclerView Performance

The app uses RecyclerView to display medication lists, today's medications, vitamins, and reminders. RecyclerView is appropriate because it efficiently reuses item views instead of creating new views for every record. For further optimization, DiffUtil can be added to adapters to improve list update performance and avoid unnecessary calls to `notifyDataSetChanged()`.

8.3. Configuration Changes

ViewModels are used to preserve UI-related data during configuration changes such as screen rotation. Because fragments observe LiveData from ViewModels, the UI can automatically update when Room data changes. This reduces the chance of data loss when the device rotates.

8.4. Memory and Lifecycle

ViewBinding is used safely by setting the binding object to null in `onDestroyView()`. This helps prevent memory leaks in fragments. Observers are attached using `viewLifecycleOwner`, which ensures they are tied to the fragment view lifecycle.

9. Code Quality and Android Best Practices

The project follows several Android development best practices required for a maintainable mobile application.

9.1. Kotlin and XML

The app is implemented using Kotlin and XML layouts. Jetpack Compose is not used. XML layouts are separated from Kotlin logic, which makes the interface easier to maintain and modify.

9.2. ViewBinding

The app uses ViewBinding to access views safely. This avoids direct `findViewById()` calls and reduces the chance of runtime ID errors. Each fragment inflates its binding object in `onCreateView()` and clears it in `onDestroyView()`.

9.3. MVVM Separation

The application avoids direct database access from UI components where possible. Fragments communicate with ViewModels, ViewModels call repositories, and repositories interact with DAOs. This separation makes the project easier to debug and explain.

9.4. Room and Coroutines

Database insert, update, and delete operations are executed on background threads using Kotlin coroutines and `Dispatchers.IO`. This prevents UI freezing and follows Android best practices for local database operations.

9.5. Navigation Component

The Navigation Component is used to define and manage fragment transitions. This centralizes navigation logic in `nav_graph.xml` and makes the app flow easier to understand.

9.6. SharedPreferences for Session Data

SharedPreferences are used to store simple session information after login, such as the current user's ID, full name, and email. This allows the app to identify the logged-in user across fragments without repeatedly querying the database.

10. Challenges Encountered

Several challenges were encountered during the development process.

10.1. Room Integration

Initially, Room entities, DAOs, and repositories existed, but some screens still used hard-coded or in-memory data. This meant the database structure existed but was not fully connected to the UI. The solution was to connect fragments to ViewModels, ViewModels to repositories, and repositories to Room DAOs.

10.2. User-Specific Data

A major challenge was separating data between users. At first, medications were stored globally, so different users could potentially see the same medication records. This was fixed by adding `userId` to `MedicationEntity` and filtering medication queries by the logged-in user's ID.

10.3. Foreign Key Safety

Reminders depend on medications through the `medicationId` foreign key. Allowing users to type medication names manually could create invalid records. The solution was to use a spinner that displays only existing medications from Room, ensuring that each reminder is linked to a real medication.

10.4. Navigation Bar Design

The bottom navigation bar needed to be fixed across the main screens while hidden from screens such as login and register. Implementing it in `MainActivity` solved this problem because it prevented duplicate navigation code inside fragments.

10.5. Theme Compatibility

Material components such as `MaterialCardView` require a compatible Material theme. A crash occurred when the app attempted to inflate Material components without the correct theme. The solution was to use a `MaterialComponents-compatible` app theme.

10.6. Database Versioning

Adding new tables and columns required increasing the Room database version. During development, `fallbackToDestructiveMigration()` was used to avoid migration crashes. This is acceptable during testing but should be replaced with real migrations in a production application.

11. Conclusion

Remedi is a native Android medication management application designed to help users organize their medications and reminders. The project follows the MVVM architecture and uses Room Database for local persistence. The app supports user registration, login, personalized medication storage, reminder creation, dashboard navigation, and profile access.

The most important improvement in the database design is the connection between users and medications through `userId`. This allows each user to have private medication data. The project also demonstrates the use of Room relationships, DAO queries, repositories, ViewModels, LiveData, ViewBinding, and the Navigation Component. Future improvements could include real notification alarms, complete dose history actions, password hashing, DiffUtil support, and formal Room migrations.

12. GitHub REPO URL

<https://github.com/RokayaRadwan/Remedi-Medical-Tracker-App>